

1 /*

2 Queue Interface:

3 --> The Queue interface of the Java collections framework provides the
4 functionality of the queue data structure.

5 --> It extends the Collection interface.

6 --> Since the Queue is an interface, we cannot provide the direct
7 implementation of it.

8 --> In order to use the functionalities of Queue, we need to use classes
9 that implement it:

10 1. LinkedList

11 2. PriorityQueue

12 3. ArrayDeque

13 --> The Queue interface is also extended by various subinterfaces:

14 1. Deque

15 2. BlockingQueue

16 3. BlockingDeque

17 --> In queues, elements are stored and accessed in First In, First Out
18 manner.

19 --> That is, elements are added from the behind and removed from the
20 front.

21 NOTES:

22 • It was introduced in JDK 5.0 version.

23 • It is a direct child interface to Collection Interface.

24 • It able to arrange all the elements as per FIFO [First In First Out],
25 but, it is possible to

26 change this algorithm as per our requirement.

27 • It able to allow duplicate elements.

28 • It is not following Insertion order.

29 • It is following Sorting order.

30 • It will not allow null values, if we add null value then JVM will rise
31 an Exception like

32 java.lang.NullPointerException.

33 • It will not allow heterogeneous elements, if we add heterogeneous
34 elements then JVM

35 will rise an exception like java.lang.ClassCastException.

36 • It able to allow only homogeneous elements.

37 • It able to allow comparable objects by default, if we add non
38 comparable objects then

39 JVM will rise an exception like java.lang.ClassCastException.

40 • If we want to add non comparable objects then we have to use
41 Comparator.

42 • It able to manage all elements prior to process.

43 --> Methods of Queue:

44 -> The Queue interface includes all the methods of the Collection
interface.

-> It is because Collection is the super interface of Queue.

-> Some of the commonly used methods of the Queue interface are:

1. `add()` - Inserts the specified element into the queue. If the task is successful, `add()` returns `true`, if not it throws an exception.
2. `offer()` - Inserts the specified element into the queue. If the task is successful, `offer()` returns `true`, if not it returns `false`.
3. `element()` - Returns the head of the queue. Throws an exception if the queue is empty.
4. `peek()` - Returns the head of the queue. Returns `null` if the queue is empty.
5. `remove()` - Returns and removes the head of the queue. Throws an exception if the queue is empty.
6. `poll()` - Returns and removes the head of the queue. Returns `null` if the queue is empty.

-----> PriorityQueue
<-----

- It was introduced in JDK 5.0 version.
- It is not Legacy Collection.
- It is a direct implementation class to Queue interface.
- It able to arrange all the elements prior to processing on the basis of the priorities.
- It able to allow duplicate elements.
- It is not following Insertion order.
- It is following Sorting order.
- It will not allow null values, if we add null value then JVM will rise an Exception like
`java.lang.NullPointerException`.
- It will not allow heterogeneous elements, if we add heterogeneous elements then JVM
will rise an exception like `java.lang.ClassCastException`.
- It able to allow only homogeneous elements.
- It able to allow comparable objects by default, if we add non comparable objects then
JVM will rise an exception like `java.lang.ClassCastException`.
- If we want to add non comparable objects then we have to use `Comparator`.
- Its initial capacity 11 elements.
- It is not synchronized .
- No method is synchronized in `PriorityQueue`.
- It allows more than one thread at a time to access data.
- It follows parallel execution.
- It able to reduce application execution time.
- It able to improve application performance.
- It is not giving guarantee for Data consistency.
- It is not threadsafe.

PriorityQueue Constructors:

1. `public PriorityQueue()` - creates empty `PriorityQueue` Object
Ex: `PriorityQueue p = new PriorityQueue();`

2. `public PriorityQueue(int capacity)` - creates empty queue with specified capacity.

```

87         3. public PriorityQueue (Collection c) - created PriorityQueue
           object with all the elements of the
88             specified collection.
89
90     */
91
92     import java.util.*;
93
94     public class Collection7 {
95         public static void main(String[] args) {
96
97             // Implementing PriorityQueue Class
98
99             ArrayList<Integer> al = new ArrayList<>();
100             al.add(30);
101             al.add(20);
102             al.add(10);
103             al.add(40);
104             System.out.println("ArrayList is: "+al);
105
106             Queue q1 = new PriorityQueue<>();
107             System.out.println(q1);
108             System.out.println(q1.size());
109
110             PriorityQueue<Integer> pq1 = new PriorityQueue<>(al);
111             System.out.println(pq1);
112             System.out.println(pq1.size());
113
114             PriorityQueue<Integer> pq2 = new PriorityQueue<>(10);
115
116             // 1. public void offer (Object obj) - insert specified element
           to Queue.
117
118             q1.offer(12);
119             q1.offer(2);
120             q1.offer(10);
121             // q1.offer("14"); // R.E: Class Cast Exception
122             q1.offer(9);
123             q1.offer(10);
124             q1.offer(8);
125             q1.offer(18);
126             // q1.offer(null); // R.E: NullPointerException
127
128             System.out.println(q1);
129
130             // 2. public Object peek() - It can be used to return head
           element of the Queue.
131
132             System.out.println("Use of peek() method: " + q1.peek());
133
134             // 3. public Object element() - It can be used to return head
           element of the Queue.
135
136             System.out.println("Use of element() method: " + q1.element());
137

```

```

138      /*
139      NOTE:
140      --> If we access peek() method on an empty Queue then peek()
141      will return
142      "null" value.
143      --> If we access element() method on an empty Queue then
144      element() method
145      will rise an exception like
146      java.util.NoSuchElementException.
147      */
148
149      // System.out.println(pq2.peek()); --> null
150      // System.out.println(pq2.element()); --> R.E.:
151      NoSuchElementException
152
153      // 4. public Object poll() - used to return and remove head
154      element from Queue.
155
156      System.out.println(q1.poll());
157      System.out.println(q1);
158
159      // 5. public Object remove() - used to return and remove head
160      element from Queue.
161
162      System.out.println(q1.remove());
163      System.out.println(q1);
164
165      /*
166      NOTE:
167      --> If we access poll() method on an empty Queue then poll()
168      method will
169      return "null" value.
170      --> If we access remove() method on an empty Queue then remove()
171      method will
172      rise an exception like "java.util.NoSuchElementException".
173      */
174
175      // System.out.println(pq2.poll()); --> null
176      // System.out.println(pq2.remove()); --> R.E.:
177      java.util.NoSuchElementException
178
179      // 6. Iterator iterator()
180
181      Iterator itr = q1.iterator();
182      while (itr.hasNext()) {
183          System.out.println(itr.next());
184      }
185
186      }
187
188      }
189
190      }

```